

第 11 章 查找和排序

查找和排序都是计算机科学中的典型问题，有着极为广泛的应用。本章将讨论几个主要的查找算法和排序算法。读者应认真领会各种查找算法和排序算法的特点、基本思想、基于的数据结构及效率，熟练掌握常用算法的实现，以便在实际中根据不同情况、不同要求选择合适的算法。

11.1 查找

查找（也称为检索）就是在一组同类型的数据元素的集合中找出满足指定条件的元素。若查找成功，则返回找到元素所在的位置，否则返回一个表示查找失败的标志。通常，把待查找的数据元素的集合称为查找表。常见的查找条件有：

- ①查找已找到元素的下一个元素；
- ②根据元素的逻辑序号查找元素；
- ③查找关键字等于给定值的元素。

本节只讨论按关键字的查找。首先介绍关键字的概念。所谓关键字是指数据元素中的一个或多个数据项值，它可以唯一标识一个数据元素。例如，在职工情况表中，职工号就是关键字。

在实际应用中可使用的查找算法较多，衡量一个查找算法好坏的标准主要有时间复杂度和空间复杂度。一般来说，各种查找算法需要的辅助空间都很少，因此，人们更关心的是算法的时间复杂度。在实际查找中，最基本的操作是用给定值与元素的关键字进行比较。这就是说，查找算法的主要时间是花费在“比较”上面，为此引入了平均查找长度的概念作为评价查找算法时间性能的依据。平均查找长度（ASL）的数学定义是

$$ASL = \sum_{i=1}^n P_i C_i$$

式中 n ——查找表的长度；

P_i ——查找第 i 个元素的概率（一般认为，对每个元素的查找概率相等，所以 P_i 的值可取为 $1/n$ ）；

C_i ——查找到第 i 个元素时需要的比较次数。

下面介绍几种常用的查找算法。

11.1.1 顺序查找

如果把查找表中的数据元素组织到一个线性表中，并用顺序方式或链接方式对它们进行存储，则实现查找最简单的方法就是顺序查找。顺序查找的基本思想是：用给定值依次与线性表中每个元素的关键字进行比较。如果某个元素的关键字与给定值相同，则查找成功；如果找遍全表也没有发现满足条件的元素，则查找失败。

在 12.2.2 和 12.2.3 节所给出的顺序表类 SeqList 和单链表类 Chain 中，成员函数 Search() 就是用顺序查找方法实现查找的。下面分析一下顺序查找的平均查找长度。

在顺序查找情况下，找到第 i 个元素时，给定值已与 i 个元素的关键字进行了比较，因此 $C_i=i$ 。在等概率情况下，顺序查找的平均查找长度为：

$$ASL = \sum_{i=1}^n P_i C_i = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2} = O(n)$$

由此可见，在用顺序查找方法完成查找时，每进行一次成功查找需要的平均比较次数约为表长度的一半，因此，它的效率较低。

11.1.2 二分查找

二分查找也称为折半查找，是一种效率较高的查找算法。在实现二分查找时，对查找表仍以线性表形式组织，并且要满足以下两个条件：

①必须以顺序方式存储线性表；

②线性表中所有数据元素必须按照关键字有序排列（在以下讨论中，假设元素按关键字递增顺序排列）。

二分查找的基本思想是：将给定值与处于顺序表“中间位置”上的元素的关键字进行比较，若相等则查找成功，若给定值大于关键字则在表的后半部分继续进行二分查找，否则在表的前半部分继续进行二分查找。如此进行下去直至找到满足条件的元素，或当前查找区域为空，查找失败。下面举例说明二分查找的查找过程。

设在一维数组 A 中存放了一组有序的整数 { -7, 3, 5, 8, 12, 16, 23, 33, 55 }。用变量 key 存放要查找的值 (33)；变量 low 存放查找区域低端元素的下标；变量 high 存放查找区域高端元素的下标；变量 mid 指示“中间位置”。mid 的值可用式 $mid = \lfloor (low + high) / 2 \rfloor$ 求得。用二分查找方法在数组 A 中查找与 key 值相等的元素的过程如图 11.1 所示。

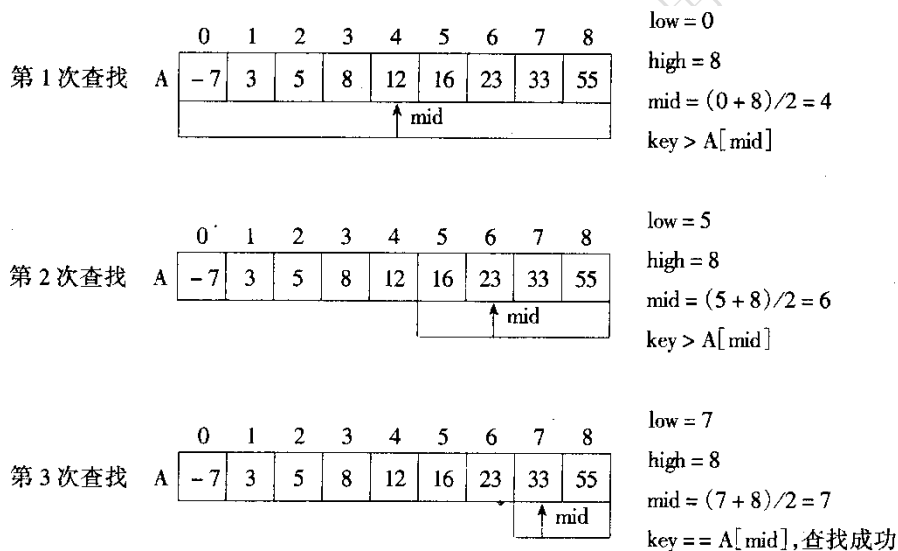


图 11.1 二分查找的查找过程

下面给出实现二分查找的 C++ 函数模板。

例 11.1 二分查找函数模板及其测试程序。

```
#include <iostream.h>
template <typename T>
int BinSearch(T A[], int low, int high, T key){
    int mid;
    while(low <= high){
        mid = (low + high) / 2;
        if(key == A[mid]) return mid;    //查找成功，返回元素的下标
        if(key < A[mid]) high = mid - 1; //到表的前半部分查找
        else low = mid + 1;             //到表的后半部分查找
    }
    return -1;                          //查找失败，返回失败标志-1
}
```

```

}

void main(){
    int a[]={1,2,3,4,5,6,7,8},k=7, i;
    i=BinSearch(a,0,7,k);
    if(i==-1)cout<<"没找到! \n";
    else cout<<i<<endl;
}

```

二分查找的优点是效率高。因为在实现二分查找时，每经过一次不相等比较后，查找范围将缩小一半。在等概率情况下，二分查找的平均查找长度为

$$ASL = \sum_{i=1}^n P_i C_i = \frac{1}{n} \sum_{j=1}^k j \times 2^{j-1} = \frac{n+1}{n} \log_2(n+1) - 1 = O(\log_2 n)$$

为方便起见，这里设 $n=2^k-1$ 。

显然，二分查找的效率较顺序查找高得多，但为换取这个效率所付出的代价是，要将表中元素按关键字有序排列，而排序是一种很费时的运算。另外，二分查找只适用于线性表的顺序存储。为保持表的有序性，在顺序表中插入和删除元素必然导致元素的大量移动。因此，二分查找特别适用于那种一经建立就很少改动而又经常需要查找的线性表。

11.1.3 分块查找

分块查找又称索引顺序查找。在此查找方式中，查找表仍以线性表形式组织，并且要求把待查找的线性表分成若干块。在每一块内，元素的存放是任意的，但在块与块之间元素的存放是有序的，即前一块中任意一个元素的关键字值都必须小于（或大于）后一块中所有元素的关键字值。另外，在实现分块查找时还要求建立一个索引表，索引表中的每个索引项对应于一块。一个索引项至少应含有两部分信息：一是对应块中最大的关键字值；二是指向对应块中第一个元素的指针。图 11.2 是一个索引表结构的示意图。

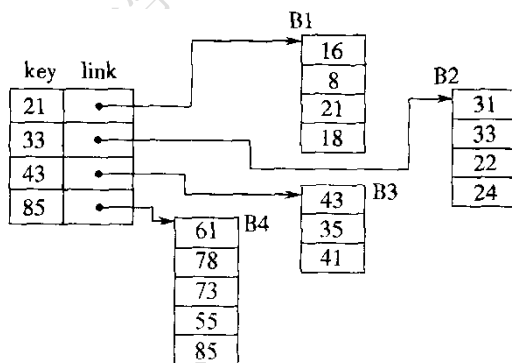


图 11.2 索引表结构示意图

分块查找的查找过程分为两步：首先在索引表中查找，确定出要查找的元素应该在哪儿一块中，然后在已确定的那一块内进行顺序查找。由此可见，分块查找的平均查找长度应由两部分组成，即

$$ASL = ASL_b + ASL_w$$

式中， ASL_b 为在索引表中进行查找的平均查找长度； ASL_w 为在块中进行查找的平均查找长度。

一般情况下，为进行分块查找，可将长度为 n 的线性表均匀地划分成 b 块，每块中有 s 个元素，即 $b = n / s$ 。假定对表中每个元素的查找概率相等，则查找某一块的概率为 $1 / b$ ，查找块中某个元素的概率为 $1 / s$ 。

若以顺序方式查找索引表，则分块查找的平均查找长度为：

$$ASL = \frac{1}{b} \sum_{j=1}^b j + \frac{1}{s} \sum_{i=1}^s i = \frac{b+1}{2} + \frac{s+1}{2} = \frac{n+s^2}{2s} + 1$$

当 $s = \sqrt{n}$ 时，ASL 取得最小值。这就是说，当 b 个块中每一块内元素的个数接近 $\sqrt{n}$ 时，平均查找长度最小。

如果以顺序方式存储索引表，那么对索引表的查找还可以采用二分查找法。在此情况下，分块查找的平均查找长度为：

$$ASL \approx \log_2 \left(\frac{n}{s} + 1 \right) + \frac{s}{2}$$

分块查找的优点是，由于块内元素是任意存放的，所以插入或删除运算不会造成元素的大量移动。分块查找的主要代价是增加了存放索引表的辅助空间及初始时对线性表分块排序的运算。另外，大量的插入和删除运算可能会导致各块中元素的数目很不均匀，这会降低查找速度。

11.1.4 二叉排序树查找

可以采用二叉排序树（或称为二叉搜索树）作为查找表中数据元素的组织结构。在二叉排序树中进行查找时，查找效率可以与二分查找相媲美，实现二叉排序树中结点插入和删除运算也比较灵活，需要移动的结点很少，可避免二分查找的不足。

1. 二叉排序树的定义

定义 二叉排序树或者是一棵空二叉树，或者是具有以下性质的二叉树：

- ①若它的左子树非空，则左子树上所有结点的值均小于根结点的值；
- ②若它的右子树非空，则右子树上所有结点的值均大于或等于根结点的值；
- ③左、右子树本身又各是一棵二叉排序树。

图 11.3 所示为一棵二叉排序树。

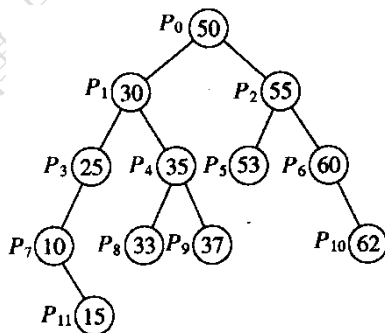


图 11.3 二叉排序树示例

2. 二叉排序树的插入操作

向二叉排序树中插入一个新结点的过程是：若二叉排序树为空，则新结点为二叉排序树的根结点；否则将新结点的关键字值和根结点的关键字值进行比较，若小于根结点的值，则将新结点插入到左子树中去，否则，插入到右子树中去。显然，这个插入过程是一个递归过程。设有整数序列 { 4 7, 2 3, 5 6, 1 5, 2 6, 8 9 }，将其中的值依次插入二叉树后得到结果如图 11.4 所示。在 10.2.5 节给出的二叉树类中，插入算法就是按以上过程实现的，

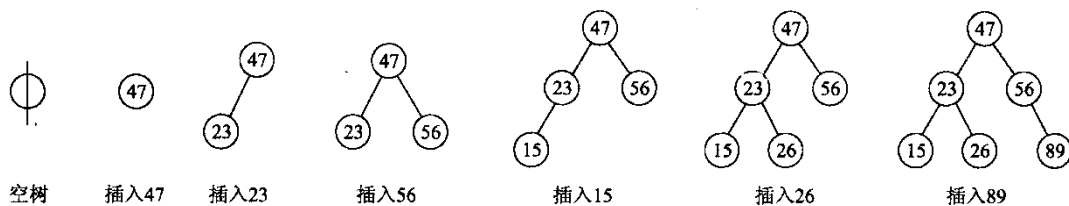


图 11.4 二叉排序树的构造过程

3. 二叉排序树的删除操作

从二叉排序树中删除一个结点的过程比较复杂, 一个结点被删除后, 必须保证余下的结点仍然构成一棵二叉排序树。下面分两种情况讨论。

(1) 被删除的结点至少有一棵空子树。

若要删除图 11.3 中值为 25 或 60 的结点, 只需使它们的那棵非空子树的根成为其双亲结点的相应子女即可, 图 11.5a 是删除 25 和 60 后得到的二叉排序树。如果被删除的结点是叶子, 则只需将其双亲结点的相应指针置为空。如果被删除的结点是根, 则其子女结点将成为根。

(2) 被删除的结点有两棵非空的子树

首先找到被删除结点在中序遍历序列中的直接后继结点(它一定在被删除结点的右子树中), 用此后继结点的值取代被删除结点的值, 然后删除此后继结点。由于被删除的后继结点的左子树一定是空子树, 所以删除后继结点的过程同 (1)。删除图 11.5a 中结点 50 后得到的二叉排序树如图 11.5b 所示。还有一种删除方法是, 用被删除结点在中序遍历序列中的直接前驱结点(该结点的右子树也一定为空)代替被删除的结点, 然后删除这个前驱结点。

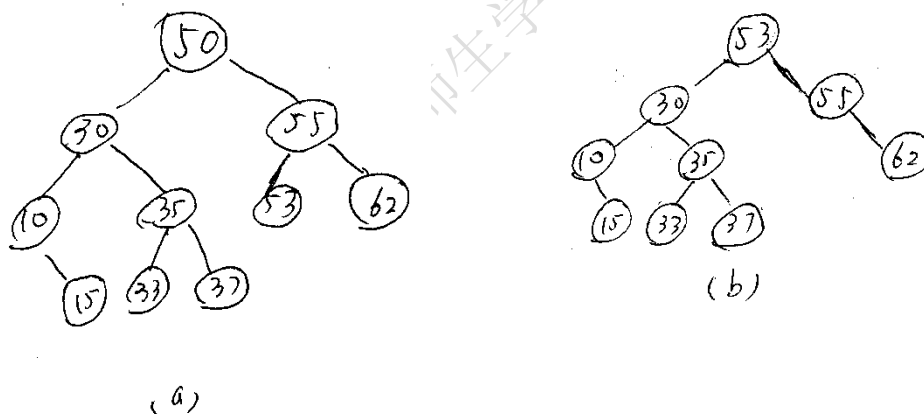


图 11.5 二叉排序树的删除

(a) 删除 25、60; (b) 删除 50

4. 二叉排序树的查找操作

对二叉排序树进行中序遍历可以得到一个数据元素由小到大的有序序列。由此可见, 对一个无序序列可以通过构造一棵二叉排序树使其变成有序序列。构造二叉排序树的过程即为对无序序列进行排序的过程。二叉排序树的名称就是由此而来的。

既然二叉排序树可以看成是一个有序表, 因此, 在二叉排序树中的查找类似于二分查找。查找过程是一个递归过程。当二叉排序树不为空时, 将给定值与根结点的值进行比较, 若相等则查找成功。如果给定值小于根结点的值, 则在根结点的左子树中继续查找, 否则在根结点的右子树中继续查找。当待查找的二叉排序树为空时, 查找失败。例如, 在图 11.3 所示的二叉排序树中, 查找与给定值 35 相同的结点的查找过程是: ①用 35 与根结点 P_0 进行比较, 因 $35 < 50$, 所以转到 P_0 的左子树进行查找; ②用 35 与左子树的根结点 P_1 比较, 因 $35 > 30$,

转到 P_1 的右子树；③用 35 与 P_4 比较，因为相等，故查找成功。由以上讨论可以看出，在二叉排序树中查找成功时走过的是一条从根结点到所寻找结点的一条路径，因此，二叉排序树查找的平均查找长度取决于二叉排序树的深度。当二叉排序树每个结点的左、右子树的高度之差不超过 1 时，二叉排序树查找的平均查找长度与二分查找相同，即 $ASL = O(\log_2 n)$ 。

通常把每个结点的左、右子树的深度至多相差 1 的二叉树称为平衡二叉树。

值得注意的是，二叉排序树是动态生成的，它的形态与各结点插入二叉排序树时的先后顺序有关。当把一组有序值依次插入到一棵二叉排序树时，生成的二叉排序树将蜕变成一棵单支树。例如，由整数序列 { 1 5, 2 3, 2 6, 4 7, 5 6, 8 9 } 生成的二叉排序树就是一棵单支树，如图 11.5 所示。

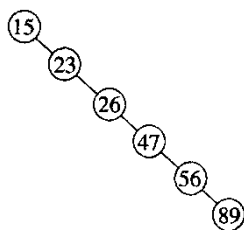


图 11.6 单支树

显然，在单支树中查找时，平均查找长度与顺序查找相同。在构造二叉排序树的过程中，可以采取“平衡化”处理以生成平衡二叉排序树。具体做法在此不详细介绍。

11.1.5 哈希查找

前面介绍的查找算法都是通过比较关键字进行的，在某些应用中，这些算法因为处理速度慢而不能满足需要。哈希（Hashing）查找法（或称散列查找法）在理想情况下完成查找操作的时间复杂度可达到 $O(1)$ 。

在讨论哈希查找之前，先介绍实现哈希查找时对查找表必须采取的组织方式——哈希表。

1. 哈希表

哈希表是一种存储方式。它的存储思想是：以数据元素的关键字 k 为自变量，通过一定的函数关系 h （称为哈希函数或散列函数）计算出对应的函数值 $h(k)$ ，把这个值解释为数据元素的存储地址并把数据元素存储到相应的存储单元内。 $h(k)$ 也称为哈希地址或散列地址。从数学观点看，哈希函数实际上是关键字到存储单元地址的映射。通常，把按上述思想实现的存储结构称为哈希表或散列表。

一般来说，哈希函数是一个压缩映象函数，由它算出的散列表地址集合通常比数据元素关键字集合要小得多，这样就有可能导致不同关键字经散列函数计算后得到的散列地址是相同的，即 $k_i \neq k_j (i \neq j)$ ，但 $h(k_i) = h(k_j)$ 。把这种现象称为冲突，发生冲突的两个关键字 k_i 和 k_j 称为同义词。

在建立一个哈希表之前须解决的问题主要有：

- ①构造一个好的哈希函数，使出现冲突的机会尽可能少；
- ②设计一个有效的解决冲突的办法。

2. 哈希函数的构造方法

在构造哈希函数时需要考虑以下几点：

- ①哈希函数的定义域必须包括要存储的数据元素的全部关键字，而如果散列表允许有 m 个地址，则哈希函数的值域必须在 0 到 $m-1$ 之间；
- ②由哈希函数计算出的地址应均匀地分布在散列地址空间内，即若 k 是从关键字集合中随机抽取的一个关键字，则哈希函数应以同等概率取 0 到 $m-1$ 中的每一个值；
- ③哈希函数应该是简单的，能在较短的时间内计算出结果。

根据关键字结构和分布的不同,可构造出不同的哈希函数。下面介绍几种常用的关于整数类型关键字的哈希函数。

(1) 直接定位法

取关键字的某个线性函数作为哈希函数,即 $h(k)=ak+b$ 。其中, k 为关键字, a 、 b 为常数。

(2) 数学分析法

当关键字的位数比存储区域地址码的位数多时,可以取关键字中位值分布比较均匀的若干位作为哈希函数的值。这种方法适合于所有关键字为已知的情况。例如,假设查找表中有 80 个数据元素,每个元素的关键字为 8 位十进制整数;哈希表的长度为 100,也就是说可以使用两位十进制数组成哈希地址。不失一般性,下面给出 8 个元素的关键字值进行分析:

$k_1=61317602$ $k_2=61326875$ $k_3=62739628$ $k_4=61343634$
 $k_5=62706816$ $k_6=62774638$ $k_7=61381262$ $k_8=61394220$

分析以上 8 个关键字可知,从左边数的第 1、2、3、6 位的位值比较集中,它们不适宜作哈希地址,而其余的 4、5、7、8 位的位值分布比较均匀,可任选其中的两位作为哈希地址,不妨选 7、8 位。这样,这 8 个关键字对应的哈希地址集合为 {02, 75, 28, 34, 16, 38, 62, 20}。

(3) 除留余数法

选择一个适当的正整数 p (p 通常选为不大于哈希表长度 m 的最大素数),用 p 去除关键字,取其余数作为哈希地址,即 $h(k)=k\%p$ ($p \leq m$)。

3. 解决冲突的方法

一个好的哈希函数可以使发生冲突的可能性尽可能少,但它不能完全避免冲突。因此,确定一个解决冲突的方法是建立哈希表时不可缺少的。解决冲突的方法主要有两种,即开放地址法和链地址法。

(1) 开放地址法

用开放地址法解决冲突的做法是,当冲突发生时形成一个地址序列,按序列顺序逐个检查各地址单元,直至找到一个未被占用的单元为止(称该单元的地址为开放地址),将发生冲突的数据元素存入该地址单元中。

最简单的探查地址序列是线性探查序列。假设发生冲突的地址是 d ,哈希表存储区域的大小为 m ,则线性探查地址序列为

$$d+1, d+2, \dots, m-1, 0, 1, \dots, d-1$$

以上探查序列也可以表示为

$$h_i=(d+i)\%m \quad (i=1, 2, \dots, m-1)$$

设哈希表 HT 的长度是 11,其初始状态为空,如果使用的哈希函数是 $h(k)=k\%11$ 并按线性探查法处理冲突,则将下面的一组关键字 {54, 77, 94, 89, 14, 45, 76} 依次存入 HT 后,HT 的状态如图 11.7 所示。

0	1	2	3	4	5	6	7	8	9	10
77	89	45	14	76		94				54

图 11.7 用线性探查法建立哈希表

除线性探查序列外还有一种常用的探查序列叫平方探查序列或叫二次探查序列。仍设发生冲突的地址为 d ,哈希表的长度为 m ,则平方探查的地址序列为:

$$(d+1^2)\%m, (d-1^2)\%m, (d+2^2)\%m, (d-2^2)\%m, \dots$$

平方探查序列也可以表示为

$$h_i = (d + d_i) \% m \quad (d_i = 1^2, -1^2, 2^2, -2^2, 3, \dots, \pm k^2 \ (k \leq m / 2))$$

(2) 链地址法

用链地址法处理冲突的基本思想是：将所有关键字是同义词的数据元素存储到同一个单链表中，而哈希表中的每个存储单元中不再存放数据元素而是存放相应的同义词单链表的头指针。例如，已知一组关键字为 { 5 4, 7 7, 9 4, 8 9, 1 4, 4 5, 7 6 }，如果按哈希函数 $h(k)=k\%11$ 和链地址法处理冲突，则哈希表的存储结构如图 11.8 所示。

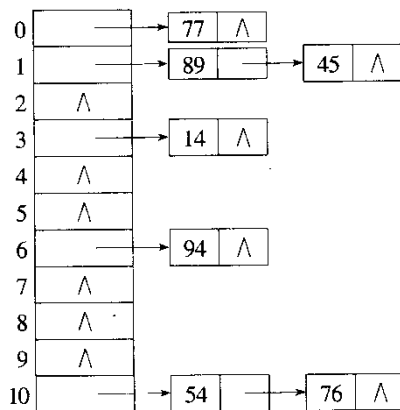


图 11.8 用链地址法解决冲突的哈希表存储结构

4. 哈希查找

在哈希表中进行查找的过程和哈希表的构造过程基本一致。对给定值 k ，以在建立哈希表时使用的哈希函数为映射函数求得一哈希地址。若哈希表中此地址位置上没有元素，则查找失败；否则，将此位置上元素的关键字与 k 进行比较，若相等则查找成功，否则根据构造哈希表时设定的处理冲突的方法得到“下一个地址”再进行查找，直至某个位置为空或某位置上元素的关键字等于 k 为止。

哈希查找的平均查找长度不直接依赖于查找表中元素的个数，而是与装填因子有关。哈希表的装填因子

$$\alpha = \frac{\text{哈希表中实际装入元素的个数}}{\text{哈希表的长度}}$$

α 标志着哈希表的装满程度。一般情况下， α 应小于 1。 α 越大，在哈希表中存入的数据元素越多，当存入新的元素时发生冲突的可能性就越大，而在查找时需要进行比较的关键字的个数也就越多。关于哈希查找的平均查找长度与 α 的关系本书不做详细讨论。

11.2 排序

排序又称为分类，是数据处理中经常使用的一种运算。排序的依据是排序码，而排序码是指数据元素的一个或多个数据项，排序码可以是关键字，也可以是其他若干个数据项的组合。

对 n 个元素组成的序列 $S=\{e_0, e_1, \dots, e_{n-1}\}$ ，按排序码非递减（或非递增）的顺序重新排列元素的过程称为排序。如果在待排序的元素序列中，存在着多个排序码相同的元素，按照某种排序算法排序后，这些元素的相对位置仍保持不变，则称该排序算法是稳定的，否则就是不稳定的。

排序的方法很多，应用也很广泛。如果整个排序过程都在计算机内存中进行，则称为内排序。如果排序过程中不仅需要使用内存，而且还要使用外存，则称为外排序。限于篇幅，本节只讨论内排序。

排序算法种类繁多,要想简单地判断出哪一种算法最好很困难。评价一个排序算法好坏的标准主要有两条:一是执行算法所需要的时间;二是执行算法所需要的附加空间。另外,算法本身的复杂程度也是需要考虑的一个因素。排序算法所需要的附加空间一般都不太大,因此,在这方面矛盾并不突出。排序的时间开销主要用算法执行中排序码的比较次数和元素的移动次数衡量。

下面介绍几种主要的内排序算法。为叙述方便,在以下的讨论中假定被排序的元素仅由排序码组成,它们以顺序方式存储在一维数组中,排序按排序码非递减顺序进行。

11.2.1 插入排序

设待排序的元素序列是 $S=\{e_0, e_1, \dots, e_{n-1}\}$,插入排序的基本思想是:把元素 $e_i(0 \leq i < n)$ 依次插入到由 S 中前 i 个元素组成的已经排好序的序列 $\{e_0, e_1, \dots, e_{i-1}\}$ 的适当位置上,插入后 S 的前 $i+1$ 个元素仍为有序。

在已排好序的元素序列中寻找要插入的元素的位置可以有多种方法,寻找方法不同,形成的插入排序算法也就不同。下面介绍两种常用的插入排序算法——直接插入排序和二分插入排序。

1. 直接插入排序

直接插入排序是以顺序查找的办法找到要插入的元素在已排好序的元素序列中应处的位置。排序的具体步骤是:初始时,已排好序的元素序列中只有一个元素 $e[0]$,下面要把 $e[1], e[2], \dots, e[n-1]$ 依次插入到已排好序的序列当中。在插入 $e[i](i=1,2,3, \dots, n-1)$ 时,先将 $e[i]$ 暂存于变量 $temp$ 中,以空出存储 $e[i]$ 的空间,然后将 $temp$ 依次与 $e[i-1], e[i-2], \dots$ 进行比较,将比 $temp$ 大的元素后移一个位置,如果发现某个元素 $e[j] \leq temp (0 \leq j < i)$,则说明 $e[j]$ 之后的 $e[j+1]$ 即为插入位置,因为比较过程已使 $e[i-1], e[i-2], \dots, e[j+1]$ 都依次向后移动了一个位置,所以把 $temp$ 存入 $e[j+1]$ 即可。如果这样的 j 不存在, $e[0]$ 即为插入位置,将 $temp$ 存入 $e[0]$ 。

例 11.2 直接插入排序函数模板及其测试程序。

```
#include <iostream.h>
template <typename T>
void InsertionSort(T e[],int n){    //对 e[0]~e[n-1]排序
    int i,j;
    T temp;
    for(i=1;i<n;i++){
        temp=e[i];
        for(j=i;j>0 && temp<e[j-1];j--) e[j]=e[j-1];
        e[j]=temp;
    }
}

template <typename T>
void PrintList(T list[],int n){
    for(int i=0;i<n;i++) cout<<list[i]<<" ";
    cout<<endl;
}

void main(){
    int a[]={38,58,13,15,51,27,10,19,12,86,49,67,84,60,25};
    cout<<"原序列: \n";
    PrintList(a,15);
}
```

```

InsertionSort(a,15);
cout<<endl<<"排序后的序列：\n";
PrintList(a,15);
}

```

程序执行结果：

原序列：

38 58 13 15 51 27 10 19 12 86 49 67 84 60 25

排序后的序列：

10 12 13 15 19 25 27 38 49 51 58 60 67 84 86

直接插入排序算法由两重循环组成：外层循环表示要进行 $n-1$ 个元素的插入；内层循环实现插入一个元素时的比较和元素后移。如果在初始状态下元素序列已经排好序（称为正序排列），则内层循环每执行一次仅需进行一次比较，总的比较次数是 $n-1$ 次，而元素总的移动次数是 $2(n-1)$ 。如果在初始状态下元素序列为逆序排列，则需要的比较次数和元素移动次数最大，均为 $O(n^2)$ 量级。

由此可见，直接插入排序算法消耗的时间与待排序元素序列的初态密切相关。当初态为正序排列时，算法的时间复杂度为 $O(n)$ ；当初态为逆序排列时，算法的时间复杂度为 $O(n^2)$ 。可以证明，直接插入排序算法的平均时间复杂度为 $O(n^2)$ 。此外，直接插入排序是一种稳定的排序算法。对于一个长度较短、接近有序的序列，直接插入排序是十分有效的。

2. 二分插入排序

因为在插入 $e[i]$ 时， $e[0], e[1], \dots, e[i-1]$ 是一个有序序列，所以可以用二分查找法寻找 $e[i]$ 的插入位置，从而达到减少比较次数的目的。

例 11.3 二分插入排序的函数模板。

```

template <typename T>
void BinInsertionSort(T e[],int n){           //对 e[0]~e[n-1]排序
    int i,j,temp;
    int low,high,mid;
    for(i=1;i<n;i++){
        temp=e[i];
        low=0;high=i-1;
        while(low<=high){                     //利用二分查找找出插入位置
            mid=(low+high)/2;
            if(temp<e[mid]) high=mid-1;
            else low=mid+1;
        }
        for(j=i-1;j>=low;j--) e[j+1]=e[j];     //元素后移
        e[low]=temp;                           //插入
    }
}

```

与直接插入排序相比，二分插入排序仅减少了对排序码的比较次数，而元素的移动次数不会改变。因此，二分插入排序算法的平均时间复杂度仍为 $O(n^2)$ 。二分插入排序算法是稳定的。

11.2.2 选择排序

选择排序的基本思想是：每次从待排序的元素序列中选择一个最小的元素，将其附加到已排好序的元素序列的后面，直至全部元素排好序为止。显然，在初始时，已排好序的元素

序列为空，待排序的元素序列中包括了需要排序的所有元素。下面介绍三种选择排序算法。

1. 直接选择排序

直接选择排序用逐个比较的办法从待排序的元素序列中选出最小的元素。其排序过程是，依次对 $i(i=0, 1, 2, \dots, n-2)$ 执行如下的选择和交换步骤：在元素序列 $e[i], e[i+1], \dots, e[n-1]$ 中选出最小的元素 $e[k]$ ；当 $k \neq i$ 时，交换 $e[i]$ 与 $e[k]$ 的值。经上述 $n-1$ 次的选择和交换后，排序工作即已完成。

例 11.4 直接选择排序函数模板。

```
template <typename T>
void SelectionSort(T e[], int n){    //对 e[0]~e[n-1]排序
    int i, j, k, temp;
    for(i=0; i<n-1; i++){
        k=i;                        //假定 e[i]最小
        for(j=i+1; j<n; j++) if(e[j]<e[k]) k=j;
        if(k!=i){
            temp=e[i];
            e[i]=e[k];
            e[k]=temp;
        }
    }
}
```

在直接选择排序中，元素的比较次数是一个固定值。它与元素序列的初始分布无关，只与序列中元素的个数有关。第 i 次选择排序码最小的元素是在子表 $e[i+1]$ 到 $e[n-1]$ 中进行，需要的比较次数为

$$n-1-(i+1)+1=n-i-1$$

排序所进行的总的比较次数为

$$\sum_{i=0}^{n-2} (n-1-i) = \frac{1}{2}n(n-1) = O(n^2)$$

由此可见，以排序码比较次数衡量的算法时间复杂度为 $O(n^2)$ ，而以元素交换次数衡量的算法时间复杂度为 $O(n)$ ，直接选择排序算法的平均时间复杂度为 $O(n^2)$ 。直接选择排序算法是不稳定的。

直接选择排序在进行后一次比较选择时没有用到前一次比较的结果，这会造成一些重复的比较，从而使算法的时间性能降低。为避免重复的比较，可对算法做相应的改进，树形选择排序（或称为锦标赛排序）就是改进后的一种选择排序算法。

2. 树形选择排序

树形选择排序的思想与体育比赛中的淘汰赛类似，首先对 n 个待排序元素的排序码两两进行比较，得到 $\lceil n/2 \rceil$ 个比较的优胜者（排序码较小者），然后对这些优胜者再进行两两比较，如此反复，直至选出排序码最小的元素，如图 11.8 所示。由于每次两两比较的结果是使排序码较小的一方作为优胜者上升到双亲结点的位置，所以也把图 11.8 所示的树称为胜者树。在下一步选择排序码为次小的元素时，只需把树当中与已选出元素对应的叶结点的值设置为最大（ ∞ ），并从该叶结点开始和其兄弟结点进行比较，修改从该叶结点到根结点路径上各结点的值，即可选出排序码次小的元素。

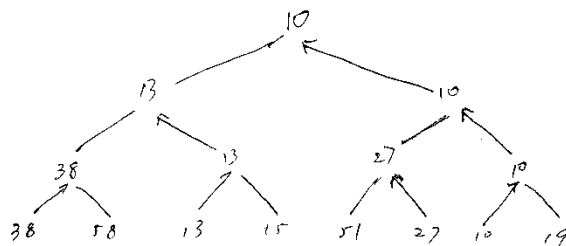


图 11.9 胜者树

在进行树形选择排序时，除第一次选择排序码最小的元素需要进行 $n-1$ 次比较外，选择排序码次小的元素、再次小的元素，...，所需要的比较次数均为 $O(\log_2 n)$ ，总的比较次数为 $O(n \log_2 n)$ ，算法的时间复杂度也为 $O(n \log_2 n)$ 。虽然树形选择排序具有较好的时间性能，但为存放胜者树需要附加额外的空间。树形选择排序是稳定的。

为弥补树形选择排序的不足，J. Williams 提出了另外一种选择排序方法，它就是堆排序。

3. 堆排序

堆排序使用了一种称为“堆”的数据结构，堆是一棵完全二叉树，其中每个分支结点的值均小于或等于左、右子女的值。显然，在一个堆中，位于堆顶位置的结点（即完全二叉树的根结点）的值是最小的。

堆排序的基本思想是：用需要排序的元素的排序码构造一个堆（称为建堆），这样就可以找到排序码最小的元素，将其取出，然后用剩下的排序码继续建堆。如此反复进行直至全部元素有序。

堆排序的关键步骤有两个：一是建立初始堆；二是在取出堆顶后把剩余的结点调整为堆。

先来看如何建立初始堆。首先把所有元素的排序码组织到一棵完全二叉树中。一般情况下，该完全二叉树不是一个堆，但它的所有以叶结点为根的子树都满足堆的定义。下面以这些已经是堆的子树（它们全是叶结点）为基础进一步把整个完全二叉树调整为堆。具体过程是：从完全二叉树中结点编号排在最后的那个分支结点 $k_m (m = \lfloor (n-2)/2 \rfloor)$ 开始，逐步地

把以 k_m 、 k_{m-1} 、 k_{m-2} ... 为根的子树建成堆，最后，当以 k_0 (k_0 是完全二叉树的根结点) 为根的完全二叉树也是堆时，整个建堆过程也就完成了。考虑以 k_i 为根的子树，设 k_i 的左、右子树均已堆。为将该子树也调整为堆，只需将 k_i 与其左子女 k_{2i+1} 和右子女 k_{2i+2} 进行比较。如果 $k_i \leq k_{2i+1}$ 和 $k_i \leq k_{2i+2}$ ，则以 k_i 为根的子树已经是堆。否则，将 k_i 与其左、右子女中最小的那个结点（不妨设为 k_{2i+1} ）交换位置。交换后，在以 k_i 为根的子树中，除了它的左子树外，其余部分均满足堆的定义，而以 k_{2i+1} （其值由于与 k_i 交换已经发生了变化）为根的子树的左、右子树原来已经是堆，这样可使用前述过程进一步将以 k_{2i+1} 为根的子树调整为堆。如此一层层地递推下去，可最终完成建堆工作。图 11.10 说明了对整数集合 $\{93, 35, 29, 45, 17, 48, 82, 76\}$ 进行建堆的过程。由于 $n=8$ ，所以建堆工作从编号为 3 的分支结点开始。

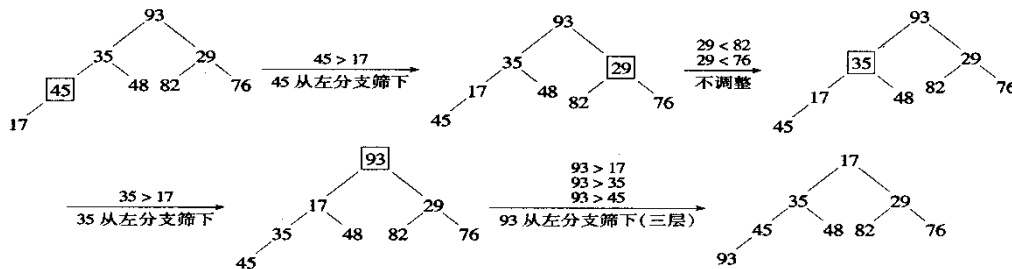


图 11.10 将完全二叉树自下而上地调整为堆

堆建成后,从堆顶取走值最小的结点,然后将最后一个结点 k_{n-1} 提升到根结点的位置上,得到一棵新的完全二叉树。虽然这棵二叉树可能不是堆,但其左、右子树都是堆。因此,当把此二叉树调整为堆时,只需从根结点开始自上而下地进行调整就可以了。

在实际编写程序实现建堆时,可以把完全二叉树以顺序方式存储到一个一维数组中,根据 10.2 节给出的性质 5,很容易找到根结点、分支结点和叶结点,以及特定结点的双亲和其左、右子女。按照前述过程在此一维数组中调整就可以得到堆。实现堆排序的程序从略。

堆排序算法的时间复杂度为 $O(n\log_2 n)$,元素的初始排列对排序时间影响不大,但由于建立初始堆的时间花费是 $O(n)$,所以如果需要排序的元素数目较少一般并不提倡使用堆排序算法,但如果元素数目较多,堆排序算法还是很有效的。堆排序算法是不稳定的。

11.2.3 交换排序

交换排序的基本思想是:两两比较待排序元素的排序码,交换不满足顺序要求的元素,直到所有元素有序。下面介绍两种交换排序算法,即冒泡排序和快速排序。

1. 冒泡排序

设待排序的元素序列为 $S=\{e_0, e_1, \dots, e_{n-1}\}$,冒泡排序的实现过程是:从序列的一端(这里假定为低端)对相邻的两个元素(e_0 和 e_1 , e_1 和 e_2 , $\dots$, e_{n-2} 和 e_{n-1})依次比较它们的排序码,若不符合排序的顺序要求,就将相应的两个元素互换,此过程称为一趟冒泡排序。其最明显的效果是使得排序码值最大的元素被交换到了最后一个元素的位置上。假设在一趟冒泡排序时,从 e_{j+1} 以后没有发生元素的互换,则说明 e_{j+1} 以后的元素是已经排好序的,下面只需要在 e_0 到 e_j 范围内进行新一趟的冒泡排序。最多经过 $n-1$ 趟冒泡排序,排序工作即可完成。在下面实现冒泡排序的 C++ 模板函数中,用变量 `lastExchangeIndex` 记录在一趟冒泡排序过程中最后一次交换的元素的下标,它在每趟排序之前被置为 0。整个冒泡排序工作在 `lastExchangeIndex` 为 0 时结束。

例 11.7 实现冒泡排序的函数模板。

```
template <typename T>
void BubbleSort(T e[],int n){ //对 e[0]~e[n-1]排序
    int i,j;
    int lastExchangeIndex;
    T temp;
    i=n-1; //i 是待排序子表中最后一个元素的下标
    while(i>0){
        lastExchangeIndex=0;
        for(j=0;j<i;j++){
            if(e[j+1]<e[j]){
                temp=e[j];
                e[j]=e[j+1];
                e[j+1]=temp;
                lastExchangeIndex=j;
            }
            i=lastExchangeIndex;
        }
    }
}
```

在冒泡排序算法中,由于在每趟冒泡时追踪记录下了最后一个交换元素的下标,因此可以避免一些重复的比较,提高了算法在某些特殊情况下的执行效率。值得注意的是,若待排序元素序列已经有序,则排序工作经一趟处理就可结束。此时,对元素排序码的比较次数取

为最小值 $n-1$, 且没有元素的互换。也就是说, 冒泡排序在最好情况下的时间复杂度为 $O(n)$ 。若待排序元素序列为逆序, 则需要进行 $n-1$ 趟冒泡。每趟要进行 $n-i-1$ 次排序码的比较和 $n-i-1$ 次元素的互换 ($0 \leq i \leq n-2$)。在此情况下, 比较次数和元素的互换次数达到最大为

$$\sum_{i=0}^{n-2} (n-i-1) = \frac{1}{2}n(n-1) = O(n^2)。因此, 冒泡排序的最坏时间复杂度为 $O(n^2)$ 。它的平均$$

时间复杂度也是 $O(n^2)$ 。冒泡排序算法是稳定的。

2. 快速排序

快速排序又称分区交换排序。基本思想是: 从待排序的 n 个元素中任取一个元素, 以该元素为基准, 用交换的方法将剩下的元素分成两组。第 1 组中各元素的排序码均小于或等于基准元素的排序码; 第 2 组中各元素的排序码均大于基准元素的排序码。把基准元素放到两组元素之间 (这也是基准元素在最后排好序的元素序列中的最终位置)。以上过程称为一趟快速排序 (或一次划分)。对划分出的两组元素重复上述过程, 直到所有元素都排列到最终位置为止。

下面举例说明实现一趟快速排序的方法。设在一维数组 A 中存放了 10 个待排序的整数值, 如图 11.11(a)所示。设置变量 low 存放最低端元素的下标 ($low=0$); 变量 $high$ 存放最高端元素的下标 ($high=9$); 变量 mid 存放中间元素的下标。 mid 的值可用 $mid = \lfloor (low + high) / 2 \rfloor$

算得。选择 $A[mid]=52$ 作为基准, 把数组中其他元素划分为两组: 位于数组低端的低端组 S_l (其中的所有元素均小于等于基准); 位于数组高端的高端组 S_h (其中的所有元素均大于基准)。由于基准最终将出现在 S_l 的末端之后, 所以可暂时将基准移到数组的最低端, 即让 $A[mid]$ 与 $A[0]$ (即 $A[low]$) 互换。为确定出 S_l 和 S_h 中的元素, 引入下标变量 $scanUp$ (初值设为 $low+1$) 和 $scanDown$ (初值设为 $high$), 见图 11.11 (b)。

首先, 通过使 $scanUp$ 每次增 1, 向右逐个扫描数组中的元素。若扫描到的元素 $A[scanUp]$ 大于基准或 $scanUp$ 大于 $scanDown$, 则停止扫描。接下来, 通过使 $scanDown$ 每次减 1, 向左逐个扫描数组中的元素。若扫描到的元素 $A[scanDown]$ 小于等于基准, 则停止扫描。此时, 如果 $scanUp < scanDown$, 则让 $A[scanUp]$ 与 $A[scanDown]$ 互换, 然后继续上述扫描过程。否则, 让 $A[0]$ 与 $A[scanDown]$ 互换。至此, 对数组中的元素已按要求划分为两组, 且选择的基准值已经处于它在排序序列的最终位置上, 见图 11.11 (c) 和图 11.11 (d)。

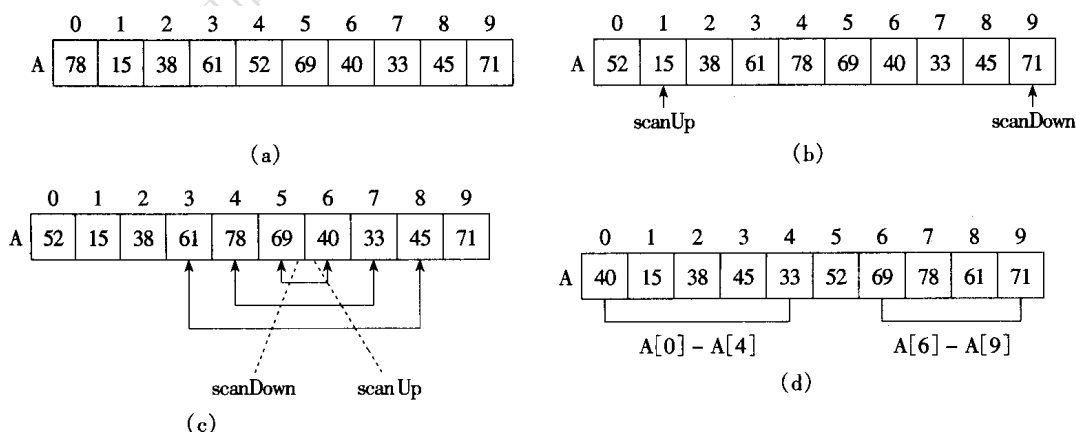


图 11.11 快速排序第 1 趟排序过程示例

对经划分后得到的两个组 $S_l(A[0] \sim A[4])$ 和 $S_h(A[6] \sim A[9])$ 进行与上述方法相同的处理, 直至最终完成对整个序列的排序。下例给出实现快速排序的 C++ 函数模板, 它是用递归方法实现的。

例 11.8 实现快速排序的函数模板。

```
template <typename T>
void Swap(T &x,T &y){
    T temp=x;
    x=y;
    y=temp;
}
template <typename T>
void QuickSort(T A[],int low,int high){    //对 A[low]~A[high]排序
    T pivot;
    int scanUp,scanDown;
    int mid;

    if(high-low<=0)
        return;
    else
        if(high-low==1){
            if(A[high]<A[low])
                Swap(A[low],A[high]);
            return;
        }
    mid=(low+high)/2;
    pivot=A[mid];
    Swap(A[mid],A[low]);
    scanUp=low+1;
    scanDown=high;
    do{
        while(scanUp<=scanDown && A[scanUp]<=pivot) scanUp++;
        while(pivot<A[scanDown]) scanDown--;
        if(scanUp<scanDown)
            Swap(A[scanUp],A[scanDown]);
    }while(scanUp<scanDown);
    A[low]=A[scanDown];
    A[scanDown]=pivot;
    if(low<scanDown-1)
        QuickSort(A,low,scanDown-1);
    if(scanDown+1<high)
        QuickSort(A,scanDown+1,high);
}
```

对快速排序的性能作出一般性分析比较困难,这里只讨论两种极端情况,即在最好情况下和最坏情况下算法的时间复杂度。快速排序最好的情况取决于每次划分时所选的基准,如果这个基准恰好处于排序序列的中间(即基准最后把原来的表划分成两个长度大致相等的子表)是最好的。为讨论方便,设表的长度 $n=2^k(k=\log_2 n)$ 。在第 1 趟排序中共进行了 $n-1$ 次比较,排序的结果产生了两个大小约为 $n/2$ 的子表;在第 2 趟排序中要处理两个子表,对每个

子表大约要进行 $n/2$ 次比较, 总的比较次数为 $2(n/2)=n$; 第 3 趟排序要处理四个子表, 对每个子表需进行约 $n/4$ 次比较, 总的比较次数为 $4(n/4)=n$ 。依此类推。这种分裂处理在进行 k 次后停止, 此时各子表的长度为 1。如此说来, 在最好情况下快速排序需要进行的比较次数大约是

$$n+2(n/2)+4(4/2)+\dots+n(n/n)=n+n+n+\dots+n=nk=n\log_2 n$$

这种最好的情况实际是在元素序列初始时已经排好序时发生的。

在快速排序中, 最坏的情况发生在基准总是落在有一个元素的子表中, 而其他元素在另一个子表中。此时, 第 1 趟排序需做 n 次比较, 而第 2 趟排序需做 $n-1$ 次比较……总的比较次数为

$$n+(n-1)+(n-2)+\dots+2=n(n+1)/2-1$$

因为快速排序时元素的移动次数不会大于比较次数, 所以快速排序最坏的时间复杂度为 $O(n^2)$, 最好的时间复杂度为 $O(n\log_2 n)$ 。可以证明, 快速排序的平均时间复杂度也是 $O(n\log_2 n)$ 。它是目前基于比较的排序算法中速度最快的, 快速排序因此得名。快速排序算法是不稳定的。对快速排序还可以进行一些改进, 在此不详细讨论。

11.3 例题

例 11.13 在 11.1.4 节叙述了对二叉排序树的查找操作, 下面给出其实现程序。

可在 13.2.5 节给出的二叉树类的 public 部分增加以下实现查找的成员函数的原型声明:

```
Node *Find(const int &k);
```

该成员函数在二叉树中查找与给定值 k 相等的结点, 如果查找成功, 则返回该结点的地址, 否则返回空。其实现为:

```
Node *BinTree::Find(const int &k){
    Node *p=root;
    while(p){
        if(k==p->data)return p;
        if(k<p->data)p=p->lchild;
        else p=p->rchild;
    }
    return 0;
}
```

下面是对该函数进行测试的程序。

```
#include "bintree.h"
void main(){
    BinTree intBTree;
    int x[12]={47,23,56,15,26,89,12,61,55,31,12,99};
    for(int i=0;i<12;i++) intBTree.insertNode(x[i]);
    Node *p;
    p=intBTree.Find(99);
    if(p) cout<<p->GetData()<<endl;
    else cout<<"没找到! \n";
}
```

例 11.14 在单链表中实现直接插入排序。

可以在 9.2.3 节给出的单链表类中加入以下实现直接插入排序的公有成员函数。测试程序请读者自己编写。


```

void Chain::InsertionSort(){
    if (length<=1) return;
    Node *current,          //指向要插入的结点
        *previous,         //指向要插入结点的直接前驱
        *p,*q;
    previous=head->next;
    current=previous->next;
    while (current!= 0) {
        p = head;
        q = head->next;
        while (current->data>=q->data&&q!=current){        //寻找插入位置
            p=q;
            q=p->next;
        }
        if (q!=current) {        //插入到 p 与 q 之间
            previous->next = current->next;
            p->next = current;
            current->next=q;
            current=previous->next;
        }
        else{
            previous=current;
            current=current->next;
        }
    }
}

```

练习 11

一、单项选择题

- 顺序查找的平均查找长度为 _____ 。
A) n B) $n/2$ C) $(n+1)/2$ D) $(n-1)/2$
- 对有 10 个元素的有序顺序表进行二分查找时，最大和最小比较次数分别是 _____ 。
A) 10 和 1 B) 5 和 1 C) 4 和 1 D) 4 和 0
- 分块查找时，在某一块内进行查找必须是 _____ 。
A) 二分查找 B) 顺序查找 C) 二分查找或顺序查找 D) 散列查找
- 在关键字随机分布的情况下，二叉排序树查找的平均查找长度与二分查找的平均查找长度量级相当。但在二叉排序树和二分查找的查找表中，插入和删除操作的实现效率前者比后者 _____ 。
A) 低 B) 高 C) 相当 D) 不能确定
- 设哈希函数 $h(k)=k \% 13$ ，哈希表地址范围为 $0 \sim 12$ 。现在，地址为 4, 5, 6, 7 的单元已被占用，其余地址为空，如果用平方探查开放地址法解决冲突，则关键字 45 的哈希地址是 _____ 。
A) 10 B) 9 C) 10 D) 7
- 在下列排序算法中，排序时间不受数据初始状态影响的是 _____ 。
A) 堆排序 B) 冒泡排序 C) 直接选择排序 D) 快速排序
- 在以下排序方法中，从待排序序列中依次取出元素与已排序序列（初始时空）中的元素做比较，将其放到已排序序列的正确位置上，称为 _____ ；从待排序序列中挑选元素，并将其放入已排序序列的一端，称为 _____ ；依次将每两个相邻的有序表合并成一个有序表的排序方法称为 _____ ；当两个元素比较出现反序时就相互交换位置的排序方法称为 _____ 。
A) 选择排序 B) 归并排序 C) 插入排序 D) 交换排序

二、简答题

- 分别画出在有序表 {5, 12, 24, 29, 41, 53, 66} 中进行二分查找时，查找关键字等于 41、24 和 33 的过程。
- 由 6 个结点构造的二叉排序树的最大深度和最小深度各是多少？
- 已知长度为 12 的整数序列 { 6 0 , 7 5 , 7 0 , 9 9 , 1 5 , 3 , 1 0 , 3 0 , 3 8 , 5 9 , 6 2 , 3 4 }，试按给定顺序构造一棵二叉排序树，并说明对 $k=99$ 的查找过程。若对二叉排序树中各结点的查找概率相等，则该二叉排序树的平均查找长度是多少？
- 从二叉排序树中删除一个结点，然后再插入这个结点，所得到的二叉排序树与原来是否相同，举例说明你的结论。
- 设哈希函数 $h(k)=k \% 13$ ，哈希表地址范围为 $0 \sim 12$ 。已知关键字序列为 19、14、23、01、68、20、84、27、55、11、10、79。试画出用线性探查开放地址法和链地址法解决冲突时哈希表的存储结构。
- 假定有 n 个互为同义词的关键字，用线性探查法把它们加入到哈希表中，请问需要多少次探查？
- 设哈希表存储在数组 $HT[m]$ ，用开放地址法解决冲突，若要加入关键字为 k_2 的新元素，计算出的哈希地址为 i ($0 \leq i \leq m-1$)，而 $HT[i]$ 已有元素，其键值为 k_1 ，请问 k_1 和 k_2 一定是同义词吗？为什么？
- 什么是排序算法的稳定性？
- 设待排序元素的排序码集合为 { 11, 4, 18, 33, 29, 9, 18, 21, 5, ... }

1 9 }, 分别写出用下列排序方法每次排序后的结果, 并说明排序码的比较次数是多少?

- (1) 直接插入排序;
- (2) 直接选择排序;
- (3) 冒泡排序;
- (4) 快速排序;
- (5) 二路归并排序;
- (6) 基数排序。

10. 对上题给出的排序码序列, 说明用堆排序时形成初始堆的过程和每选出一个排序码后重新建堆的过程。

11. 在实现快速排序算法时, 可将基准选为 $A[\text{low}]$ 而不是 $A[\text{mid}]$, 此时算法的时间复杂度仍为 $O(n\log_2 n)$, 但最坏情况与选 $A[\text{mid}]$ 时不同, 你能说出最坏情况在何时发生吗?

三、程序填空

以下是实现双向冒泡排序的 C++ 函数。所谓双向冒泡排序是指以正反两个方向交替进行冒泡排序, 即第 1 趟把排序码最大的元素移动到最末尾, 第 2 趟把排序码最小的元素移动到最前面。如此反复进行, 直至排序完成。

```
void swap(int &x, int &y) {
    int temp=x;
    x=y;
    y=temp;
}

void DBubble(int A[], int n) { //对 A[0]~A[n-1]排序
    int d, b1, b2, le, k;
    d=0; //d 为 0, 表示正向冒泡, 为 1 表示反向冒泡
    b1=0; b2=n-1; //b1, b2 分别记录冒泡范围的低端和高端下标
    while( _____ ) {
        if (d==0) {
            le=b1; //le 记录被交换元素的下标
            for(k=b1; k<b2; k++)
                if (A[k+1]<A[k]) {
                    swap(A[k], A[k+1]);
                    le=k;
                }
            b2= _____ ;
            d=1;
        }
        else {
            le=b2;
            for(k=b2; _____ ; k--)
                if (A[k]<A[k-1]) {
                    swap(A[k], A[k-1]);
                    le=k;
                }
            _____ =le;
        }
    }
}
```

```
        d=0;  
    }  
}  
}
```

四、算法设计题

1. 试编写递归的二分法查找算法。
2. 编写程序判断单链表中的结点是否按递增顺序排列。
3. 试编写基于单链表的直接选择排序和冒泡排序算法。

仅供疫情期间天津大学师生学习课程使用，严禁翻印！